OPEN SOURCE AUDIT REPORT

Capstone Project — Open Source Software

Chosen Package: Python (CPython)

Student Name	Basavanneppa Shiwalingappa Mugad
Registration No.	24BCY10322
Course	Open Source Software
Chosen Software	Python — CPython Distribution
Submission Date	March 2026
Report Type	Shell Scripting Capstone — Open Source Audit

Department of Computer Science & Engineering
Academic Year 2025–2026

Abstract

This report is the result of a hands-on open-source audit I carried out on the Python programming language — specifically the CPython implementation — as part of my coursework in Open Source Software. The audit involved writing five shell scripts from scratch, each targeting a different aspect of how Python exists on and interacts with a Linux-based system. Rather than just reading about open-source principles in a textbook, the goal was to actually go and look — to inspect, question, and document what Python is doing on a real machine.

Python was the obvious pick for me, not just because it's everywhere, but because it genuinely embodies what open-source software is supposed to be. The governance model, the licensing, the community-driven development process — it all holds together in a way that's worth studying carefully. Each of the five scripts was written to demonstrate a specific set of Bash scripting skills while producing output that's actually useful for understanding how Python is installed and configured on the host system.

The scripts cover system identification, package inspection, directory permission auditing, log file analysis, and manifesto generation. Taken together, they walk through the major programming constructs of Bash — variables, loops, conditionals, arrays, file I/O, user input, and text processing tools like awk and grep. The results confirmed that Python 3 is properly installed, correctly maintained, and tightly woven into the system's core infrastructure.

Table of Contents

1. Introduction to Open Source Software *4*
2. Why I Chose Python *5*
3. A Brief History of Python *6*
4. Python's Licensing — The PSF Model *7*
5. Project Overview and Script Suite *7*
6. Script 1 — sys_identity.sh: System Identity Report *8*
7. Script 2 — package_inspector.sh: FOSS Package Inspector *9*
8. Script 3 — disk_auditor.sh: Disk and Permission Auditor *10*
9. Script 4 — log_analyzer.sh: Log File Analyzer *11*
10. Script 5 — manifesto.sh: Manifesto Generator *12*
11. Bash Concepts — Cross-Script Summary *13*
12. Observations, Difficulties, and What I Learned *14*
13. Conclusion *15*
14. References *15*

1. Introduction to Open Source Software

Open source is one of those ideas that sounds simple until you actually sit with it for a while. At its most basic, it just means that the source code for a piece of software is publicly available — anyone can read it, copy it, change it, and share it. But the implications of that simplicity go pretty deep.

Think about what it means for software to be transparent. You're not just trusting a company's word that their product is secure, or that it does what they claim. You can go look. Anyone can look. And if something is broken or wrong, anyone can fix it and submit that fix back. That's a completely different relationship between users and software than what we've come to accept as normal with proprietary products.

The open-source movement didn't appear overnight — it grew out of the hacker culture of the 1970s and 80s, when software was routinely shared among researchers and academics. Richard Stallman's GNU Project in 1983 was a deliberate attempt to formalize and protect that culture as commercial software companies started locking code away behind licenses. The Linux kernel in the early 90s showed the world that a community of volunteers could build something as complex as an operating system — and build it well.

1.1 What Makes Software Truly Open Source

The Open Source Initiative lays out a specific definition that goes beyond just 'the code is visible.' True open source has to allow free redistribution, has to come with the actual source code in a usable form, and has to permit people to create derived works. It also cannot discriminate — against individuals, groups, or particular kinds of use. These aren't pedantic rules for their own sake; they're the conditions that keep software genuinely free over time.

- Source code must be available and usable — not obfuscated or compiled-only
- Anyone can redistribute freely, with or without modifications
- Modified versions and derivative works must be permitted
- No discrimination against people, communities, or fields of use
- The license applies to the software itself, not who happens to be using it

1.2 Shell Scripting as an Audit Tool

For this project, the audit instrument of choice was Bash — the shell that runs by default on almost every Linux system in existence. Shell scripting might not seem glamorous compared to Python or Java, but for system-level work there's nothing more direct. A good Bash script can query the kernel, interrogate the package manager, read log files, traverse directory trees, and interact with users — all without installing anything extra. It's the Swiss Army knife of Linux administration.

Each of the five scripts in this project was written to test something specific about the system's relationship with Python, while also demonstrating a particular cluster of Bash programming concepts. The connection between the two goals — audit purpose and learning objective — was deliberate. Every feature of each script exists because it's useful, not just because it was on a checklist.

2. Why I Chose Python

Honestly, the choice wasn't hard. When thinking about which open-source software to audit, the question wasn't 'why Python' — it was more like 'what else would even make sense?' Python is on every Linux system I've ever used. It's used by the OS itself, by package managers, by system tools. It's the language I've used most in my coursework. And it's a genuinely great example of how open source is supposed to work.

That last point mattered to me. Some software is open source technically but governed in ways that feel closed. Python isn't like that. The Python Enhancement Proposal process means anyone can suggest a change to the language, the community debates it openly, and decisions get made by an elected steering council. Guido van Rossum — who created Python in the late 80s — eventually stepped back from being the sole decision-maker and handed control over to the community. That kind of institutional maturity is rare and worth studying.

2.1 Python's Reach in the Real World

The numbers are hard to argue with. Python has been at or near the top of every major programming language ranking for several years running. The TIOBE Index, the Stack Overflow Developer Survey, the IEEE Spectrum rankings — Python appears in the top position on all of them. There are over 8 million active Python developers globally, and that number keeps climbing as more data science, machine learning, and automation work gets done in Python.

But what strikes me more than the raw numbers is where Python shows up. It's used by NASA for mission planning. Instagram's backend was built on it. Google uses it heavily internally. The machine learning frameworks that power modern AI — TensorFlow, PyTorch, scikit-learn — all expose Python APIs. A language that started as a scripting tool for system administrators ended up as the primary interface for some of the most complex software being built today.

2.2 Python on Linux Specifically

For the purposes of this audit, the most relevant fact about Python is that it lives on Linux systems at a very fundamental level. On Ubuntu 22.04, Python 3 is installed as a dependency of the OS itself. You can't remove it without breaking things. System utilities written in Python include the Software Updater, the hardware detection layer, and components of the package management infrastructure. So when we audit Python, we're really auditing a core layer of the system.

- Python 3 ships as a default package on Ubuntu, Debian, Fedora, and most modern distros
- System tools like update-manager and ubuntu-drivers are written in Python
- The /usr/bin/python3 binary is a dependency of hundreds of other packages
- Python's standard library lives in /usr/lib/python3.x — protected, root-owned space
- pip and venv allow user-level package management that doesn't touch system Python

3. A Brief History of Python

Python's origin story is a good one. Guido van Rossum started writing it in December 1989 — during a Christmas break, of all times — while working at the Centrum Wiskunde & Informatica research center in the Netherlands. He wanted something more expressive than the shell, more readable than C, and more powerful than the ABC language he'd been working with. He also wanted it to be fun to use. That last requirement turned out to matter a lot.

The name came from Monty Python's Flying Circus, which tells you something about the culture he was trying to create. Python was always meant to be a language people enjoyed working in, not just a tool they tolerated. That spirit shows up in the Zen of Python — a collection of aphorisms about the language's design philosophy that you can read by typing 'import this' in any Python interpreter.

3.1 Key Milestones in Python's Development

1991 — Python 0.9.0 posted to the alt.sources newsgroup. Even at this early stage, it had exception handling, classes with inheritance, and the core built-in types.

1994 — Python 1.0 released. Lambda expressions, map(), filter(), and reduce() were added, bringing functional programming patterns into the language.

2000 — Python 2.0 arrived with list comprehensions, cycle-detecting garbage collection, and proper Unicode support. This version dominated for nearly two decades.

2008 — Python 3.0 — a deliberate, breaking change. Print became a function. Unicode became the default string type. Division was fixed to behave the way mathematicians expect. Backwards compatibility was sacrificed for correctness.

2020 — Python 2 officially reached end-of-life after a long, carefully managed transition period. The community coordinated the migration remarkably well.

2023-25 — Python 3.11 through 3.13 brought significant performance gains — CPython is measurably faster than it was five years ago. Experimental work on removing the GIL opened new possibilities for true parallelism.

What stands out about this history isn't just the technical progression but the way the community handled it. The Python 2 to 3 transition took over a decade and required enormous coordination, but it was done transparently, with extensive documentation, compatibility tools, and clear timelines. That kind of disciplined project management on a volunteer-driven open-source project is genuinely impressive.

4. Python's Licensing — The PSF Model

Python is distributed under the Python Software Foundation License, usually abbreviated to PSF License. It's a permissive license — meaning it puts very few restrictions on what you can do with Python. You can use it commercially, embed it in proprietary software, modify it, redistribute it, and you don't have to open-source your own code just because you're using Python. The only real requirement is attribution: you have to keep the copyright notice and the license text in your distribution.

4.1 Permissive vs. Copyleft Licensing

There are two broad camps in open-source licensing: permissive and copyleft. Permissive licenses like the PSF License, MIT, and BSD say 'do what you want, just give us credit.' Copyleft licenses like the GPL say 'do what you want, but if you distribute modified versions, you have to distribute them under the same license.'

Neither approach is objectively better — they reflect different values. The GPL is designed to ensure that improvements to software stay open and benefit everyone. Permissive licenses are designed to maximize adoption, even at the cost of allowing proprietary derivatives. Python's choice of a permissive license has clearly been vindicated by its adoption: companies that might have been wary of GPL terms have built massive commercial products on Python without restriction.

4.2 Practical Consequences for Developers

- Python can be used in commercial projects without licensing fees or legal complications
- Companies can embed Python in proprietary software without disclosing their own source code
- The copyright notice must be preserved — you can't strip the attribution
- The PSF License is GPL-compatible, so Python can be bundled with GPL-licensed software
- Individual Python packages on PyPI carry their own licenses — always worth checking

One thing worth clarifying: using Python doesn't mean your own code becomes open source. That's a common misconception. The PSF License governs the Python interpreter and standard library, not code you write that happens to run on Python. Your code is yours to license however you want.

5. Project Overview and Script Suite

The project consists of five Bash shell scripts. Each one does a specific job, and together they form something like an audit pipeline — starting with 'what machine is this' and ending with 'what does open source mean to me personally.' That arc from objective system data to personal reflection was intentional.

All five scripts were written to run on a standard Debian-based Linux environment. They use only tools that are available by default on such systems — no special installations required. The one exception is that Script 4 needs a log file to analyze, so it takes that path as a command-line argument.

5.1 Script Summary

Script File	What It Does	Bash Concepts Used
sys_identity.sh	Reports kernel, distro, user, uptime, and date	Variables, command substitution, piped commands

package_inspector.sh	Checks if Python3 is installed and fetches its metadata	if-elif, case statement, dpkg/rpm queries
disk_auditor.sh	Audits permissions and sizes of key system directories	Arrays, for loop, awk, du, ls -ld
log_analyzer.sh	Counts keyword matches in a log file line-by-line	While read loop, file checks, grep, argument handling
manifesto.sh	Generates a personal open-source philosophy document	read -p, heredoc, file creation, date formatting

5.2 Environment Details

Shell	GNU Bash 5.x (/bin/bash)
Operating System	Ubuntu 22.04 LTS — Jammy Jellyfish
Package Manager	dpkg / apt
Python Version	Python 3.10.x (CPython)
Core Utilities Used	awk, grep, cut, tail, du, ls, date, uptime, dpkg
Execution	chmod +x *.sh then ./script_name.sh

6. Script 1 — sys_identity.sh: System Identity Report

```
File: sys_identity.sh
```

6.1 What This Script Does and Why It Matters

Before you can audit anything, you need to know where you are. That's the whole point of this script — it answers the first questions a sysadmin asks when sitting down at an unfamiliar terminal. What kernel is running? What distro is this? Who's logged in? How long has the machine been up?

It sounds basic, and in a way it is. But that context matters. The version of Python installed on Ubuntu 20.04 is different from the one on Fedora 38. The package manager that installed it, the default library paths, the system Python symlink — all of these depend on the OS. So collecting system identity information isn't just a warm-up exercise; it sets the stage for everything that comes after.

6.2 Bash Concepts at Work

- Scalar variable assignment — STUDENT_NAME='...' stores metadata about the report itself
- Command substitution with $(...) — the shell runs a command and uses its output as a value
- Chained piping — grep, cut, and tr are connected in sequence to clean up the distro name
- Date formatting strings — date +'%Y-%m-%d %H:%M:%S' produces a readable timestamp
- echo with string interpolation — variables drop into output strings naturally

6.3 A Closer Look at the DISTRO Line

The line that extracts the distribution name is worth unpacking because it shows how Unix tools chain together. The /etc/os-release file contains several key-value pairs about the OS. We want just the PRETTY_NAME value, but it comes with quotes around it that we need to strip:

```
DISTRO=$(cat /etc/os-release | grep -w "PRETTY_NAME" | cut -d= -f2 | tr -d '"')
```

Reading left to right: cat sends the file contents into the pipe; grep -w finds the line where PRETTY_NAME appears as a whole word; cut -d= -f2 splits on the equals sign and takes the second piece; and tr -d '"' removes the double-quote characters. Four commands, one clean variable. This is the Unix philosophy in miniature.

6.4 Sample Output

```
================================
 Open Source Audit — Basavanneppa Shiwalingappa Mugad
================================
Kernel     : 5.15.0-76-generic
Distro     : Ubuntu 22.04.2 LTS
User       : basavanneppa
Home Dir   : /home/basavanneppa
Uptime     : up 3 hours, 41 minutes
Date/Time  : 2026-03-29 14:22:10
================================
License    : This operating system is primarily covered by the GNU GPL.
Chosen     : Python
```

7. Script 2 — package_inspector.sh: FOSS Package Inspector

```
File: package_inspector.sh
```

7.1 What This Script Does

This is the script most directly connected to the audit's subject. It checks whether Python 3 is installed using whatever package manager is available on the system, then pulls out key metadata: the version number, the maintainer, and a short description. It also prints a one-liner about Python's significance in the open-source world.

One thing I wanted to get right with this script is cross-distribution compatibility. A script that only works on Ubuntu isn't very useful. So the logic first checks whether dpkg is available (which it would be on Debian-based systems), then falls back to rpm (for Red Hat-based systems), then falls back to a basic command -v check if neither package manager is present. That layered fallback approach is something you see in production-grade shell scripts constantly.

7.2 The Conditional Logic

```
if command -v dpkg &> /dev/null; then
    if dpkg -l "$PACKAGE" &>/dev/null; then
        echo "$PACKAGE is installed."
        dpkg -s "$PACKAGE" | grep -E 'Version|Maintainer|Description' | head -n 3
    else
        echo "$PACKAGE is NOT installed."
    fi
elif command -v rpm &> /dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary'
    fi
fi
```

The &>/dev/null redirects both stdout and stderr to the null device — this is how we silently check whether a command succeeds without showing any output to the user. The exit code (0 for success, non-zero for failure) is what the if statement actually tests.

7.3 The case Statement

After the package check, the script runs a case statement that pattern-matches on the package name and prints a philosophical note about it. For Python, the output is: 'Python: A language shaped entirely by community, readability, and the Zen of Python.' The case statement with a wildcard default means this script could be adapted for any package with minimal changes:

```
case $PACKAGE in
    python|python3)
        echo "Python: A language shaped entirely by community and the Zen of Python."
        ;;
    firefox)
        echo "Firefox: A nonprofit browser fighting for an open web."
        ;;
    *)
        echo "$PACKAGE: An important part of the FOSS ecosystem."
        ;;
esac
```

7.4 Expected Output

```
python3 is installed.
Version: 3.10.12-1~22.04.2
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Description: interactive high-level object-oriented language
----------------------------------------
Python: A language shaped entirely by community, readability, and the Zen of Python.
```

8. Script 3 — disk_auditor.sh: Disk and Permission Auditor

```
File: disk_auditor.sh
```

8.1 Why Permissions Matter

File permissions are one of those topics that feel boring until something goes wrong. Then they feel like the most important thing in the world. A world-writable /tmp with the sticky bit missing can be exploited for privilege escalation. A /etc/passwd file that's writable by ordinary users is a serious vulnerability. The /usr/bin directory being owned by a non-root user would be a red flag for a potential compromise.

The Disk and Permission Auditor checks a predefined list of critical system directories and reports their permissions, ownership, and disk usage. It then does the same for Python's library directory, making the connection between general system security and the specific software being audited.

8.2 Arrays and For Loops in Action

The central feature of this script is the array. Rather than hard-coding five separate checks, the directories are stored in a Bash array and the loop handles each one with the same code:

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print "Perms: "$1" | Owner: "$3":"$4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done
```

The ${DIRS[@]} expansion gives us every element of the array. The double quotes around it preserve any directory names that might contain spaces (which none of these do, but it's good practice). The -d test checks that the directory actually exists before trying to query it — a small thing that prevents the script from crashing on an unusual system.

8.3 Understanding the awk Command

The awk one-liner deserves explanation. The output of ls -ld /etc looks something like this: drwxr-xr-x 12 root root 4096 Jan 1 00:00 /etc. The fields we want are $1 (the permissions string), $3 (the owner), and $4 (the group). awk extracts them and formats them into a readable label. It's a compact but powerful use of a tool that many people underestimate.

8.4 Directory Reference

Directory	Purpose	Expected Perms	Security Note
/etc	System-wide configuration files	root:root 755	Must not be world-writable
/var/log	System and application logs	root:syslog 775	Group-writable for log daemons
/home	User home directories	root:root 755	Individual dirs should be 700 or 750
/usr/bin	Core system binaries	root:root 755	No write access for non-root users
/tmp	Temporary file space	root:root 1777	Sticky bit (1) must be set

9. Script 4 — log_analyzer.sh: Log File Analyzer

```
File: log_analyzer.sh
```

9.1 Logs Are How Systems Talk to You

If a system is having a problem and you want to understand it, the first place you go is the logs. That's true whether you're troubleshooting a web server that keeps returning 500 errors, investigating a failed login attempt, or trying to figure out why a service crashed at 3 AM. The ability to parse and analyze log files isn't optional knowledge for a sysadmin — it's foundational.

This script takes a log file path and a keyword as arguments, counts how many times the keyword appears, and then shows the last five matching lines. A built-in retry mechanism handles the case where someone accidentally passes an empty file. It's not a replacement for tools like logwatch or ELK stack, but it demonstrates the core logic that underlies all of them.

9.2 Argument Handling and Defaults

The script introduces two new concepts that haven't appeared in the earlier scripts: positional arguments and default parameter values. $1 and $2 are the first and second arguments passed on the command line. The syntax ${2:-'error'} means 'use $2, but if it's unset or empty, use the word error instead.' This is a clean way to give scripts sensible defaults:

```
LOGFILE=$1
KEYWORD=${2:-"error"}

# Three different failure modes, each checked separately:
if [ -z "$LOGFILE" ]; then        # no argument given at all
    echo "Usage: $0 <log_file> [keyword]"
    exit 1
fi
if [ ! -f "$LOGFILE" ]; then      # argument given but file doesn't exist
    echo "Error: File $LOGFILE not found."
    exit 1
fi
```

9.3 The while read Loop

The keyword counting uses a while IFS= read -r loop. This is the right way to read a file line by line in Bash — and it's worth understanding why the alternatives don't work as well:

```
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
```

```
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
```

The IFS= prefix clears the Internal Field Separator for this loop, which means leading and trailing whitespace in each line is preserved. The -r flag tells read not to interpret backslashes as escape characters. Without these two small additions, some log lines would be mangled. The < input redirection at the end feeds the file into the loop without spawning a subshell — an important efficiency detail for large files.

9.4 Usage Examples

```
# Basic usage — search for 'error' (the default keyword)
./log_analyzer.sh /var/log/syslog

# Search for a specific keyword
./log_analyzer.sh /var/log/auth.log failed

# Search for Python-related events
./log_analyzer.sh /var/log/syslog python
```

10. Script 5 — manifesto.sh: Open Source Manifesto Generator

```
File: manifesto.sh
```

10.1 The Most Personal Script

This one is different from the others. The first four scripts are purely technical — they gather data, inspect packages, audit directories, count log entries. They don't ask for your opinion. The manifesto script does. It asks you three questions, then uses your answers to write a personal declaration of open-source philosophy and save it to a file.

I included this in the project because open source isn't just a technical concept. It's a set of values, and the most interesting question isn't whether you understand what open source means academically — it's whether you've actually internalized it. Do you believe that sharing code makes the world better? Do you use open-source tools every day without thinking about it? Would you build something and give it away? The manifesto is a way to make those questions concrete.

10.2 Interactive Input and the Heredoc

The script uses read -p to prompt the user for three answers and store them in variables. Then it uses a heredoc — the cat << EOF syntax — to compose a multi-paragraph document with those variables embedded naturally. The heredoc is one of the most useful and underappreciated features of Bash:

```
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

cat << EOF > "$OUTPUT"
--- The Hacker's Vow ---
Date: $DATE
Author: $USER_NAME

As a technologist, I depend on the foundations laid by others before me.
Every day, I use tools like $TOOL to perform my work and grow my skills.
```

```
To me, freedom means $FREEDOM...
EOF
```

The > '$OUTPUT' at the end of the cat << EOF line redirects everything between EOF and the closing EOF into a file. Variables like $TOOL and $FREEDOM are expanded normally inside the heredoc, so the resulting document is personalized to whoever ran the script.

10.3 A Sample Generated Manifesto

```
--- The Hacker's Vow ---
Date: 29 March 2026
Author: shivam

As a technologist, I depend on the foundations laid by others before me.
Every day, I use tools like Python to perform my work and grow my skills.
Open source represents the spirit of collaboration where the pursuit of
knowledge outvalues corporate walls.

To me, freedom means curiosity. It means the right to look inside the
machine, understand how it ticks, and fix it if it breaks.
If I could share one creation with the world, I would build a free online
learning platform and license it openly for anyone to use and adapt.
```

11. Bash Concepts — Cross-Script Summary

One of the goals of this project was to cover the major programming constructs of Bash across the five scripts. The table below maps each concept to the scripts where it appears. Where a concept is demonstrated particularly well in a script, it appears with a check mark.

Bash Concept	Script 1	Script 2	Script 3	Script 4	Script 5
Variable Assignment	✓	✓	✓	✓	✓
Command Substitution $(...)	✓	✓	✓		✓
if / elif / else Conditionals		✓	✓	✓	
case Statement		✓			
Bash Arrays			✓		
for Loop over Array			✓		
while IFS= read -r Loop				✓	
Positional Arguments ($1, $2)				✓	
Default Parameter Values				✓	
Interactive Input (read -p)					✓
Heredoc (cat << EOF)					✓
File Creation and Writing				✓	✓
awk Field Extraction	✓	✓	✓		
grep Pattern Matching	✓	✓		✓	
Arithmetic $(( ))				✓	

12. Observations, Difficulties, and What I Learned

12.1 Things That Were Harder Than Expected

Making Scripts Work Across Distributions

The biggest practical challenge was writing scripts that would actually work across different Linux distributions without modification. Debian-based and Red Hat-based systems use completely different package managers, different default paths, and sometimes different command-line tools. The package_inspector.sh script had to handle all of this through nested conditionals and the command -v fallback. Getting that logic right without access to a Red Hat system for testing required careful reading of the documentation.

The Empty File Edge Case in log_analyzer.sh

The retry loop in the log analyzer — where the script asks if you want to specify a different file when the original turns out to be empty — was trickier to implement than it looks. The do-while pattern doesn't exist natively in Bash the way it does in C or Java, so it had to be simulated with a while loop that reads a user response. Making sure the retry logic tested both file existence and file size separately took a few iterations to get right.

stderr Suppression in disk_auditor.sh

The du -sh command generates permission errors when it encounters directories it can't read — which happens more often than you'd expect when scanning /var/log or /etc. Redirecting stderr to /dev/null with 2>/dev/null silences those errors without affecting the actual output. It's a small thing but a necessary one, and it's the kind of detail that separates a script that's pleasant to use from one that floods the terminal with noise.

12.2 What I Observed About Python on This System

- Python 3.10 is installed as a default system package, not something I installed manually — it came with the OS
- The package maintainer listed by dpkg -s is Ubuntu Developers, not the PSF — downstream distros repackage and maintain their own builds
- Python's standard library lives in /usr/lib/python3.10/, which is root-owned and has 755 permissions — readable by everyone, modifiable only by root
- The /usr/bin/python3 file is actually a symlink that points to python3.10 — this is how distros manage multiple Python versions without breaking system tools
- The /tmp directory had the sticky bit set (permissions 1777), which is correct — without it, any user could delete other users' temporary files
- Searching /var/log/syslog for 'python' returns almost no results — Python errors are typically caught by the applications themselves and logged to application-specific files, not syslog

12.3 The Bigger Picture

Working through this project changed how I think about the software on my machine. I've used Python for years without thinking much about where it came from, who maintains it, what license it's under, or how it sits inside the file system. Going through the audit process forced me to actually look at all of that — and what I found was that Python is genuinely well-engineered, carefully maintained, and deeply integrated in a way that respects the design principles of the systems it runs on.

The other thing this project reinforced is something I sort of knew intellectually but hadn't felt concretely before: the tools themselves — Bash, awk, grep, du, ls — are all open source. The infrastructure I used to audit an open-source package is itself open source. It's open source all the way down. The whole system is built on shared work, and we interact with the fruits of that work every time we open a terminal.

13. Conclusion

This project started as a coursework requirement and turned into something I found genuinely interesting to work through. Writing five shell scripts to audit Python forced me to engage with parts of the Linux system I don't normally think about — the package metadata, the directory permission structure, the log file format, the way the kernel version and distro identity are stored. All of that is more interesting than it sounds in the abstract.

Python turned out to be a great subject for this kind of audit. It's everywhere, it's well-documented, it has a clear and sensible licensing model, and its governance structure is a genuinely good example of how open-source projects can be run transparently and fairly. The PSF's stewardship of the language — including the carefully managed Python 2 to 3 transition — shows that open-source projects can be both principled and pragmatic.

The Bash scripts collectively demonstrate the key programming constructs covered in the course: variable handling, command substitution, arrays, loops, conditionals, file I/O, user interaction, and text processing. More importantly, each script does something real. The system identity report gives you genuine system information. The package inspector tells you actual package metadata. The log analyzer counts real occurrences in real log files. The manifesto generator produces a file you can actually keep.

Open source works because people share. Every line of CPython, every entry in the Python documentation, every answer on the Python mailing list — all of it came from someone who chose to give their time and knowledge freely. Auditing that software is a small way of paying attention to something that deserves more attention than it usually gets.

14. References

[1] Python Software Foundation. (2024). Python Documentation. https://docs.python.org

[2] Open Source Initiative. (2024). The Open Source Definition. https://opensource.org/osd

[3] Van Rossum, G. (1991). Python 0.9.0 — initial release to alt.sources newsgroup. Usenet Archive.

[4] Python Software Foundation. (2024). PSF License Agreement. https://docs.python.org/3/license.html

[5] GNU Project / Free Software Foundation. (2024). GNU General Public License v2. https://www.gnu.org/licenses/gpl-2.0.html

[6] The Linux Documentation Project. (2024). Advanced Bash-Scripting Guide. https://tldp.org/LDP/abs/html/

[7] Ubuntu Community. (2024). Ubuntu Package Archive — python3. https://packages.ubuntu.com/jammy/python3

[8] Stack Overflow. (2024). Developer Survey 2024 — Most Popular Technologies. https://survey.stackoverflow.co/2024/

[9] TIOBE Software BV. (2025). TIOBE Programming Community Index. https://www.tiobe.com/tiobe-index/

[10] Raymond, E. S. (2001). The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary. O'Reilly Media.

— End of Report —

Basavanneppa Shiwalingappa Mugad | Reg. No. 24BCY10322 | Open Source Software | 2025–26